# СКРИПТЫ И SQL

Как файлы проекта превращаются в таблицы и отчёты

Карта запуска: какие файлы загружают данные, строят модель, рассчитывают ответы и проверяют результат.

### Что такое скрипт и SQL

Скрипт - сохранённая последовательность действий. Файлы .sh запускают команды на компьютере: например, включают базу и отправляют в неё SQL. Файл .py - программа на Python, которая у нас загружает CSV. SQL - язык команд для базы: создать таблицу, записать строки, посчитать сумму, проверить связи.

### Как всё проходит по шагам

- 1. CSV в data/ - восемь исходных файлов.
- 2. scripts/load_raw.py загружает их в raw - восемь исходных таблиц.
- 3. 01_model/10_stg.sql создаёт stg - очищенные и типизированные копии.
- 4. 20_core_dimensions.sql и 30_core_facts.sql строят core - справочники и операции.
- 5. 40_mart.sql готовит mart - данные для анализа доставки и каталог показателей.
- 6. 90_dq.sql записывает проверки в dq.check_result. Критическая ошибка отменяет сборку.
- 7. scripts/run_answers.sh считает ответы и сохраняет CSV в 02_answers/block_a/results/.

raw, stg, core, mart и dq - области внутри PostgreSQL, похожие на папки для таблиц. Папки проекта хранят файлы с кодом, а таблицы и их строки находятся в самой базе.

### Makefile: короткие команды

- make setup: подготовить окружение и заново загрузить исходные CSV.
- make build: пересобрать stg, core, mart и dq из уже загруженного raw.
- make dq: показать результаты проверок последней сборки. Новые проверки эта команда не запускает.
- make answers: пересчитать ответы, обновить CSV и сверить контрольные цифры.
- make all: выполнить setup, build и answers по очереди.

docker-compose.yml задаёт запуск PostgreSQL и хранение базы в Docker. Отдельно можно включить pgAdmin - веб-интерфейс для просмотра базы. Makefile вызывает скрипты, а Docker обеспечивает работающий сервер базы.

# ЗАГРУЗКА ДАННЫХ

Подготовка окружения и восьми исходных таблиц

Пять файлов готовят окружение и исходные таблицы. Обычная точка входа - make setup.

### scripts/setup.sh

Проверяет Docker, наличие файлов, Python и библиотеку подключения к PostgreSQL. При необходимости вызывает download_data.sh, запускает PostgreSQL, ждёт готовности и вызывает load_raw.py. В конце показывает контрольное количество таблиц и строк. Результат - работающая база с загруженным raw. Модель core этот скрипт не собирает.

При повторном запуске исходные таблицы загружаются заново. Если core уже был собран, он остаётся прежним до make build. Параметр --reset удаляет старую базу с данными перед установкой.

### scripts/download_data.sh

Берёт список восьми имён файлов, скачивает CSV из репозитория Olist и сохраняет в data/. Уже существующий непустой файл пропускает. Этот скрипт работает с файлами на диске, таблицы базы не создаёт и не заполняет.

### scripts/load_raw.py

Проверяет, что все восемь файлов существуют и заголовки колонок соответствуют ожидаемым. По умолчанию выполняет 02_raw_ddl.sql, затем загружает CSV через COPY - массовую передачу строк в PostgreSQL. Добавляет _source_file - имя файла, и _loaded_at - время загрузки. Загрузка работает одной транзакцией: при ошибке изменения не сохраняются.

Исходные поля остаются текстовыми. Даты, деньги и города здесь не исправляются. Параметр --skip-ddl пропускает пересоздание таблиц, но данные всё равно очищаются и загружаются заново.

### scripts/init/01_init.sql

При первом создании базы контейнер выполняет этот файл: создаёт области raw, core и mart. Также задаёт, где искать таблицу, если запрос не указал область явно. Схемы stg и dq появятся позже при сборке модели.

### scripts/init/02_raw_ddl.sql

Определяет колонки восьми таблиц raw и их типы. Например, raw.orders хранит заказы, а raw.order_items - товарные строки. Все исходные колонки имеют тип text, даже даты и суммы. Служебное время загрузки имеет тип timestamptz. Перед созданием файл удаляет старые таблицы raw; загрузчик по умолчанию тоже выполняет этот файл.

# СБОРКА И ОЧИСТКА

Порядок выполнения и подготовленный слой stg

Эти файлы организуют выполнение SQL и готовят данные к основной модели.

### scripts/psql.sh

Это общий вход для отправки SQL в PostgreSQL. Использует psql на компьютере, если он установлен. Иначе запускает psql внутри контейнера Docker. Получает параметры подключения из переменных окружения и передаёт аргументы дальше. При ошибке SQL останавливает выполнение. Сам бизнес-расчётов не содержит.

### scripts/build_model.sh

Берёт все файлы 01_model/*.sql по порядку имён: 00, 10, 20, 30, 40, 90. Передаёт их в базу через psql.sh с параметром --single-transaction. Все шаги либо сохраняются вместе, либо полностью отменяются. После успешной сборки показывает dq.check_result - таблицу результатов проверок.

Цифры в начале имён задают зависимости: сначала подготовить области, затем очистить данные, затем создать справочники, операции, отчётные данные и проверки.

### 01_model/00_schemas.sql

Удаляет и создаёт заново области stg, core, mart и dq со всем их содержимым. raw сохраняется. Это означает полную пересборку подготовленных данных. При ошибке последующего шага транзакция отменит и эти удаления, вернув предыдущую рабочую модель.

### 01_model/10_stg.sql

Читает восемь таблиц raw и создаёт восемь подготовленных таблиц stg. Убирает внешние пробелы, превращает пустой текст в NULL, приводит суммы к числам и даты к типу timestamp. При повторении ключа выбирает строку с более поздним _loaded_at. Результат хранится в таблицах, поэтому очистку не нужно повторять при каждом чтении.

- clean_text: « SP » превращается в «SP», пустая строка - в NULL.
- to_numeric и to_int: текст «125.50» становится числом, «3» - целым числом.
- to_timestamp: разбирает текстовую дату и время, сохраняя их без часового пояса.
- to_zip_prefix: «1001» превращается в «01001».
- normalize_city: «sao paulo - sp» превращается в «sao paulo», email и чисто числовое значение - в NULL.

NULL означает отсутствие значения. Проверка формата отсекает часть некорректных дат, но строка с подходящим форматом и несуществующей датой может остановить сборку. Неподходящие исходные значения не всегда можно безопасно превратить в пустые.

# СПРАВОЧНИКИ И ФАКТЫ

Двенадцать основных таблиц в core

Два SQL-файла создают 12 основных таблиц: восемь справочников и четыре таблицы операций.

### 01_model/20_core_dimensions.sql

Читает подготовленные данные stg и создаёт восемь справочников:

- dim_customer: одна запись на человека по customer_unique_id, последний известный адрес и первый заказ.
- dim_seller: продавец, нормализованный город, исходный город и штат.
- dim_product: товар, категория, вес и размеры. Цена хранится в продаже, потому что может меняться.
- dim_category: категории и переводы. Номер 0 означает «Без категории», -1 - «Неизвестно».
- dim_date: каждый день 2016-2020 годов, включая дни и месяцы без заказов.
- dim_state: штаты Бразилии и их регионы.
- dim_order_status: статусы и правила, в том числе is_revenue - учитывать ли заказ в выручке.
- dim_payment_type: способы оплаты, включая not_defined - «не определён».

Записи получают ключи для связей с операциями. Для неизвестных объектов предусмотрены служебные записи. Например, продажа отсутствующего в справочнике товара получает product_sk = -1 и остаётся видна.

### 01_model/30_core_facts.sql

Читает stg и справочники core. Создаёт четыре таблицы:

- fact_order: одна строка на заказ. Даты этапов, адрес доставки на момент заказа, суммы, количество товаров, длительности и признаки проблем.
- fact_order_item: одна строка на единицу товара. Заказ, товар, продавец, цена, доставка, срок отгрузки.
- fact_payment: одна строка на платёж. Способ, сумма, номер платежа, число частей оплаты.
- fact_review: один отзыв на конкретный заказ. Оценка, текст и отметка последнего отзыва заказа.

Стоимость товаров и сумму платежей сначала считает отдельно по заказу. Иначе три товара и два платежа дали бы шесть сочетаний и повторный подсчёт денег. Последнюю оценку отзыва переносит в заказ. Заказы без товаров или платежей сохраняет с признаками has_items и has_payment.

В конце добавляет внешние ключи - проверки существования связанных записей. Строки с неизвестным товаром используют предусмотренный справочником номер -1. А платёж без существующего заказа нарушит связь, и сборка не пройдёт.

# ВИТРИНЫ И ПРОВЕРКИ

Подготовленные данные отчётов и контроль качества

Здесь появляются данные для анализа доставки и результаты самопроверок модели.

### 01_model/40_mart.sql

Читает только core. Создаёт три объекта:

- mart.v_delivery: сохранённый запрос с доставленными заказами, у которых есть дата вручения и корректная хронология. Заказы оформлены до сентября 2018. Каждая строка - один заказ. Здесь сроки, опоздание, оценка и суммы.
- mart.agg_delivery_hist: физическая таблица итогов по месяцу, штату и числу дней доставки. Например, в штате SP за январь 100 заказов ехали пять дней, 150 - шесть. Хранит количества и суммы, ускоряет отчёты.
- mart.metric_catalog: список показателей с формулой, источником и правилом расчёта по группам. Это описание расчётов, а не готовые числовые ответы на все вопросы.

Представление v_delivery выполняет запрос при чтении. Таблица agg_delivery_hist хранит уже рассчитанные итоги и обновляется при пересборке модели. Сам интерактивный экран дашборда этот файл не создаёт.

### 01_model/90_dq.sql

Создаёт dq.check_result и записывает туда 39 проверок. Для каждой есть код, описание, ожидаемое и фактическое значение, итог и время проверки.

- R01-R11: сверки источника и модели - количество строк и суммы.
- C01-C06: согласованность внутри core - суммы заказа, статусы, итоги по категориям.
- P01-P05: разбор значений и обработка повторов.
- I01-I04: ссылки на «Неизвестно» и новые значения справочников.
- B01-B10: известные особенности - нет товаров, расходятся суммы, перепутаны даты.
- M01-M03: согласованность подробных и подготовленных данных доставки.

error означает критическую проверку: несовпадение вызывает ошибку и откат всей сборки. warn сообщает о данных, требующих внимания. Для известных проблем ожидаемое значение может быть не нулевым: например, 775 заказов без товаров. Совпадение с этим числом означает, что проблема учтена, а не устранена.

# ОТВЕТЫ НА ВОПРОСЫ

Пять расчётов, выгрузки CSV и контрольные цифры

Эти SQL-файлы читают core, считают ответы и не пересоздают основную модель.

### scripts/run_answers.sh

Выполняет q*.sql в папках 02_answers/block_*/ и сохраняет CSV в results/. Из тех же запросов создаёт временные представления q01-q05 для checks.sql. После завершения они исчезают. Расхождение с контрольной цифрой завершает команду ошибкой.

### 02_answers/block_a/q01_revenue_by_month.sql

Группирует заказы по месяцу оформления. Считает оформленные и доставленные заказы, выручку без доставки. Календарь сохраняет пустой ноябрь 2016. За сентябрь 2016 - август 2018 получаем 24 строки и выручку 13 221 498,11.

### 02_answers/block_a/q02_top_sellers.sql

Суммирует доставленные товары по продавцам, добавляет город, штат, число заказов и единиц. Оставляет топ-10 по выручке. Каждому продавцу относит только его товары. У первого - 226 987,93.

### 02_answers/block_a/q03_avg_order_value.sql

Усредняет order_amount доставленных заказов: товары плюс доставка, весь заказ целиком. Также считает среднее количество строк и единиц товара. Средний чек - 159,88.

### 02_answers/block_a/q04_customers_repeat.sql

Считает заказы каждого постоянного покупателя. Вернувшийся - человек с двумя заказами или больше. Учитывает все статусы. Результат: 96 096 покупателей, вернулись 2 997, доля - 3,12%.

### 02_answers/block_a/q05_revenue_by_category.sql

Суммирует выручку доставленных товаров по категориям, считает доли. Включает «Без категории». Получаем 74 группы, их выручка в сумме совпадает с общей.

### 02_answers/block_a/checks.sql

Сверяет 12 контрольных условий: количество строк, покупателей, категорий, средний чек и суммы. Использует временные q01-q05 из run_answers.sh. Самостоятельно без них не выполняется. Запуск всей проверки - make answers.

# ОТДЕЛЬНЫЕ ЗАПУСКИ

Исследование источника, расчёты дашборда и нагрузка

Эти файлы запускаются отдельно. make all автоматически их не выполняет.

### docs/profiling/profiling.sql

Читает raw и исследует источник до построения модели. Считает строки, повторы ключей, пропуски, статусы, диапазоны дат и расхождения сумм. Его результат - таблица «область / находка / значение» в терминале. Данные не исправляет. Результаты исследования описаны в docs/01_data_profile.md.

Пример отдельного запуска:

```
bash scripts/psql.sh -f - < docs/profiling/profiling.sql
```

### 04_dashboard_concept/dashboard_queries.sql

Читает core и mart. Считает цифры концепции дашборда: долю вовремя, медиану доставки, обещанный срок, оценки по опозданию, результаты по месяцам и штатам, повторные покупки и продавцов с задержками. Выводит несколько таблиц в терминал. Новые таблицы core не создаёт, интерактивный экран не строит.

```
bash scripts/psql.sh -f - < 04_dashboard_concept/dashboard_queries.sql
```

### 05_strategy/bench/bench.sh

Создаёт отдельную тестовую базу bench, копирует туда raw и увеличивает поток заказов в указанное число раз. Номера заказов и людей меняет, товары и продавцов оставляет прежними. Затем запускает ту же модель и измеряет время сборки и типовых запросов. Запросы прогревает первым выполнением, время берёт со второго.

Результат дописывает в 05_strategy/bench/results.csv. Тестовую базу bench пересоздаёт в начале и удаляет после успешного завершения. Рабочую базу marketplace не размножает. При ошибке до финального шага тестовая база может остаться.

```
bash 05_strategy/bench/bench.sh 10
```

Число 10 означает десятикратный поток заказов. Это отдельный нагрузочный эксперимент, а не ежедневная команда получения отчётов.

# КАК ЧИТАТЬ SQL

Команды, пример запроса и места изменения правил

Названия команд описывают конкретные действия над таблицами и строками.

### Основные команды

- SELECT: выбрать поля и вычисления. FROM: указать источник.
- WHERE: отобрать строки, например доставленные заказы.
- JOIN: связать таблицы по ключу. LEFT JOIN сохраняет строки слева даже без пары справа.
- GROUP BY: сгруппировать строки, например по продавцу.
- SUM: сложить суммы. COUNT: посчитать строки. AVG: вычислить среднее.
- DISTINCT: убрать повторы результата. COUNT(DISTINCT ...): посчитать разные объекты.
- ORDER BY: отсортировать. LIMIT: ограничить число строк.
- CREATE TABLE: создать таблицу. INSERT: добавить строки. COPY: массово загрузить строки.
- CREATE VIEW: сохранить запрос. TRUNCATE: очистить таблицу. DROP: удалить объект.
- PRIMARY KEY: уникальный ключ строки. FOREIGN KEY: ссылка на существующую строку другой таблицы.
- COMMIT: сохранить транзакцию. ROLLBACK: отменить изменения. Это команды базы, отдельные от Git.

### Пример: посчитать стоимость товаров по заказу

```
SELECT order_id, SUM(price) AS items_amount
FROM core.fact_order_item
GROUP BY order_id;
```

Группируем товарные строки по заказу и складываем цены. AS items_amount называет колонку результата. Получаем одну строку на заказ. SELECT исходные данные не изменяет.

### Где менять правила

- Правило выручки: 01_model/20_core_dimensions.sql, справочник dim_order_status.
- Очистка города, даты, числа: 01_model/10_stg.sql.
- Суммы заказа, длительности и признаки проблем: 01_model/30_core_facts.sql.
- Отбор заказов в анализ доставки: 01_model/40_mart.sql, v_delivery.
- Формулировка ответа на вопрос: соответствующий q*.sql в 02_answers/.
- Проверки модели: 01_model/90_dq.sql. Проверки ответов: block_a/checks.sql.

Ответ для защиты: «Скрипты организуют запуск и загрузку. SQL строит модель, считает и проверяет отчёты. После изменения модели запускаем make build и make answers. После изменения только запроса ответа - make answers».